

Résumé Complet MongoDB

CRUD, Filtrage, Pipeline, Agrégation

AmineGR03

1^{er} février 2026

Table des matières

Opérations CRUD

Create (Insertion)

Insertion d'un document

```
1 db.collection.insertOne({
2   nom: "Dupont",
3   age: 30,
4   ville: "Paris"
5 })
```

Insertion de plusieurs documents

```
1 db.collection.insertMany([
2   {nom: "Martin", age: 25, ville: "Lyon"},
3   {nom: "Bernard", age: 35, ville: "Marseille"}
4 ])
```

Insertion avec insert()

```
1 db.collection.insert({
2   nom: "Durand",
3   age: 28,
4   ville: "Toulouse"
5 })
```

Read (Lecture)

Lecture de tous les documents

```
1 db.collection.find()
```

Lecture avec formatage

```
1 db.collection.find().pretty()
```

Lecture d'un seul document

```
1 db.collection.findOne({age: 30})
```

Projection (sélection de champs)

```
1 // Inclure seulement nom et age
2 db.collection.find({}, {nom: 1, age: 1, _id: 0})
3
4 // Exclure ville
5 db.collection.find({}, {ville: 0})
```

Update (Mise à jour)

Mise à jour d'un document

```
1 db.collection.updateOne(  
2     {nom: "Dupont"},  
3     {$set: {age: 31}}  
4 )
```

Mise à jour de plusieurs documents

```
1 db.collection.updateMany(  
2     {ville: "Paris"},  
3     {$set: {pays: "France"}}  
4 )
```

Opérateurs de mise à jour

```
1 // $set : d finir une valeur  
2 {$set: {age: 30}}  
3  
4 // $inc : incr menter  
5 {$inc: {age: 1}}  
6  
7 // $unset : supprimer un champ  
8 {$unset: {ville: ""}}  
9  
10 // $push : ajouter un tableau  
11 {$push: {hobbies: "lecture"}}  
12  
13 // $pull : retirer d'un tableau  
14 {$pull: {hobbies: "lecture"}}  
15  
16 // $addToSet : ajouter si n'existe pas  
17 {$addToSet: {tags: "important"}}
```

Replace (remplacement)

```
1 db.collection.replaceOne(  
2     {nom: "Dupont"},  
3     {nom: "Dupont", age: 32, ville: "Lyon"}  
4 )
```

Delete (Suppression)

Suppression d'un document

```
1 db.collection.deleteOne({nom: "Dupont"})
```

Suppression de plusieurs documents

```
1 db.collection.deleteMany({ville: "Paris"})
```

Suppression de tous les documents

```
1 db.collection.deleteMany({})
```

Filtrage et Requêtes

Opérateurs de Comparaison

```
1 // galit
2 db.collection.find({age: 30})
3
4 // Plus grand que
5 db.collection.find({age: {$gt: 25}})
6
7 // Plus grand ou gal
8 db.collection.find({age: {$gte: 25}})
9
10 // Plus petit que
11 db.collection.find({age: {$lt: 40}})
12
13 // Plus petit ou gal
14 db.collection.find({age: {$lte: 40}})
15
16 // Différent de
17 db.collection.find({age: {$ne: 30}})
18
19 // Dans une liste
20 db.collection.find({ville: {$in: ["Paris", "Lyon"]}})
21
22 // Pas dans une liste
23 db.collection.find({ville: {$nin: ["Paris", "Lyon"]}})
```

Opérateurs Logiques

```
1 // ET (AND)
2 db.collection.find({
3   $and: [
4     {age: {$gt: 25}},
5     {ville: "Paris"}
6   ]
7 })
8
9 // OU (OR)
10 db.collection.find({
11   $or: [
12     {age: {$lt: 30}},
13     {ville: "Lyon"}
14   ]
15 })
16
17 // NON (NOT)
18 db.collection.find({
19   $not: {age: {$gt: 30}}
20 })
21
22 // NOR
23 db.collection.find({
24   $nor: [
25     {age: {$lt: 25}},
26     {ville: "Paris"}
27   ]
28 })
```

```
28 })
```

Opérateurs pour Tableaux

```
1 // Tous les éléments du tableau correspondent
2 db.collection.find({
3     tags: {$all: ["important", "urgent"]}
4 })
5
6 // Taille du tableau
7 db.collection.find({
8     hobbies: {$size: 3}
9 })
10
11 // Éléments existent dans le tableau
12 db.collection.find({
13     hobbies: "lecture"
14 })
15
16 // Éléments à une position spécifique
17 db.collection.find({
18     "hobbies.0": "lecture"
19 })
```

Opérateurs pour Chaînes de Caractères

```
1 // Commence par
2 db.collection.find({
3     nom: {$regex: "^Dup", $options: "i"}
4 })
5
6 // Contient
7 db.collection.find({
8     nom: {$regex: "pont", $options: "i"}
9 })
10
11 // Se termine par
12 db.collection.find({
13     nom: {$regex: "ont$", $options: "i"}
14 })
```

Opérateur Exists

```
1 // Champ existe
2 db.collection.find({
3     email: {$exists: true}
4 })
5
6 // Champ n'existe pas
7 db.collection.find({
8     email: {$exists: false}
9 })
```

Opérateur Type

```
1 // Vérifier le type
2 db.collection.find({
3     age: {$type: "number"}
4 })
5
6 // Types possibles: double, string, object, array, bool, date, null, int
   , long, decimal
```

Requêtes Imbriquées

Accès aux Champs Imbriqués

```
1 // Document avec structure imbriquée
2 {
3     nom: "Dupont",
4     adresse: {
5         rue: "123 Main St",
6         ville: "Paris",
7         codePostal: "75001"
8     }
9 }
10
11 // Requête sur champ imbriqué
12 db.collection.find({
13     "adresse.ville": "Paris"
14 })
15
16 // Mise à jour d'un champ imbriqué
17 db.collection.updateOne(
18     {nom: "Dupont"},
19     {$set: {"adresse.codePostal": "75002"}}
20 )
```

Tableaux d'Objets

```
1 // Document avec tableau d'objets
2 {
3     nom: "Dupont",
4     commandes: [
5         {produit: "Livre", prix: 20},
6         {produit: "Stylo", prix: 5}
7     ]
8 }
9
10 // Recherche dans tableau d'objets
11 db.collection.find({
12     "commandes.prix": {$gt: 10}
13 })
14
15 // Recherche avec $elemMatch
16 db.collection.find({
17     commandes: {
18         $elemMatch: {
19             produit: "Livre",
20             prix: {$gt: 15}
21         }
22     }
23 })
```

Requêtes Complexes Imbriquées

```
1 // Structure complexe
2 {
```



```
3     entreprise: {
4         nom: "TechCorp",
5         employes: [
6             {
7                 nom: "Martin",
8                 departement: {
9                     nom: "IT",
10                    budget: 100000
11                }
12            }
13        ]
14    }
15 }
16
17 // Requête multi-niveaux
18 db.collection.find({
19     "entreprise.employes.departement.budget": {$gt: 50000}
20 })
21
22 // Avec $elemMatch
23 db.collection.find({
24     "entreprise.employes": {
25         $elemMatch: {
26             "departement.nom": "IT",
27             "departement.budget": {$gt: 50000}
28         }
29     }
30 })
```

Tri et Limitation

Tri

```
1 // Tri croissant
2 db.collection.find().sort({age: 1})
3
4 // Tri d croissant
5 db.collection.find().sort({age: -1})
6
7 // Tri multiple
8 db.collection.find().sort({ville: 1, age: -1})
```

Limitation

```
1 // Limiter le nombre de r sultats
2 db.collection.find().limit(10)
3
4 // Ignorer les premiers r sultats
5 db.collection.find().skip(5)
6
7 // Combinaison
8 db.collection.find()
9     .sort({age: -1})
10    .skip(10)
11    .limit(5)
```

Pipeline d'Agrégation

Introduction

Le pipeline d'agrégation permet de traiter et transformer des documents à travers plusieurs étapes.

Stages Principaux

\$match - Filtrage

```
1 db.collection.aggregate([
2   {
3     $match: {
4       age: {$gt: 25},
5       ville: "Paris"
6     }
7   }
8 ])
```

\$project - Projection et Transformation

```
1 db.collection.aggregate([
2   {
3     $project: {
4       nom: 1,
5       age: 1,
6       ageAnneeProchaine: {$add: ["$age", 1]},
7       estMajeur: {$gte: ["$age", 18]},
8       _id: 0
9     }
10  }
11 ])
```

\$group - Regroupement

```
1 db.collection.aggregate([
2   {
3     $group: {
4       _id: "$ville",
5       nombrePersonnes: {$sum: 1},
6       ageMoyen: {$avg: "$age"},
7       ageMin: {$min: "$age"},
8       ageMax: {$max: "$age"}
9     }
10  }
11 ])
```

\$sort - Tri

```
1 db.collection.aggregate([
2   {
3     $sort: {age: -1}
4   }
5 ])
```

```
5 ])
```

\$limit - Limitation

```
1 db.collection.aggregate([
2   {
3     $limit: 10
4   }
5 ])
```

\$skip - Ignorer

```
1 db.collection.aggregate([
2   {
3     $skip: 5
4   }
5 ])
```

\$unwind - Déplier un Tableau

```
1 // Avant unwind
2 {
3   nom: "Dupont",
4   hobbies: ["lecture", "sport", "musique"]
5 }
6
7 // Apr s unwind
8 db.collection.aggregate([
9   {
10    $unwind: "$hobbies"
11  }
12 ])
13
14 // R sultat: 3 documents s par s , un pour chaque hobby
```

\$lookup - Jointure

```
1 db.commandes.aggregate([
2   {
3     $lookup: {
4       from: "clients",
5       localField: "clientId",
6       foreignField: "_id",
7       as: "clientInfo"
8     }
9   }
10 ])
```

\$addFields - Ajouter des Champs

```
1 db.collection.aggregate([
2   {
3     $addFields: {
4       agePlusUn: {$add: ["$age", 1]},
5       nomComplet: {
6         $concat: ["$prenom", " ", "$nom"]
7       }
8     }
9   }
10 ])
```

\$count - Compter

```
1 db.collection.aggregate([
2   {
3     $match: {age: {$gt: 25}}
4   },
5   {
6     $count: "nombrePersonnes"
7   }
8 ])
```

Opérateurs d'Agrégation

Opérateurs Arithmétiques

```
1 // Addition
2 {$add: ["$prix", "$taxe"]}
3
4 // Soustraction
5 {$subtract: ["$prix", "$remise"]}
6
7 // Multiplication
8 {$multiply: ["$quantite", "$prixUnitaire"]}
9
10 // Division
11 {$divide: ["$total", "$nombre"]}
12
13 // Modulo
14 {$mod: ["$nombre", 2]}
15
16 // Puissance
17 {$pow: ["$base", "$exposant"]}
18
19 // Racine carrée
20 {$sqrt: "$nombre"}
21
22 // Valeur absolue
23 {$abs: "$difference"}
```

Opérateurs de Comparaison

```
1 // égal
2 {$eq: ["$a", "$b"]}
3
4 // Plus grand que
5 {$gt: ["$a", "$b"]}
6
7 // Plus grand ou égal
8 {$gte: ["$a", "$b"]}
9
10 // Plus petit que
11 {$lt: ["$a", "$b"]}
12
13 // Plus petit ou égal
14 {$lte: ["$a", "$b"]}
15
16 // Différent de
17 {$ne: ["$a", "$b"]}
18
19 // Comparaison multiple
20 {$cmp: ["$a", "$b"]} // Retourne -1, 0, ou 1
```

Opérateurs Logiques

```
1 // ET
2 {$and: [condition1, condition2]}
```

```
3
4 // OU
5 {$or: [condition1, condition2]}
6
7 // NON
8 {$not: condition}
9
10 // Condition ternaire
11 {$cond: {
12     if: condition,
13     then: valeurSiVrai,
14     else: valeurSiFaux
15 }}
```

Opérateurs pour Chaînes

```
1 // Concat nation
2 {$concat: ["$prenom", " ", "$nom"]}
3
4 // Sous-chaîne
5 {$substr: ["$texte", 0, 5]}
6
7 // Longueur
8 {$strLenCP: "$texte"}
9
10 // Convertir en minuscules
11 {$toLower: "$texte"}
12
13 // Convertir en majuscules
14 {$toUpper: "$texte"}
```

Opérateurs de Date

```
1 // Année
2 {$year: "$date"}
3
4 // Mois
5 {$month: "$date"}
6
7 // Jour
8 {$dayOfMonth: "$date"}
9
10 // Jour de la semaine
11 {$dayOfWeek: "$date"}
12
13 // Heure
14 {$hour: "$date"}
15
16 // Différence entre dates (en millisecondes)
17 {$subtract: ["$dateFin", "$dateDebut"]}
```

Opérateurs de Tableau

```
1 // Taille du tableau
2 {$size: "$tableau"}
3
4 // Filtrer un tableau
5 {$filter: {
6   input: "$tableau",
7   as: "item",
8   cond: {$gt: ["$$item.prix", 10]}
9 }}
10
11 // Prendre les n premiers éléments
12 {$slice: ["$tableau", 3]}
13
14 // Concatener des tableaux
15 {$concatArrays: ["$tableau1", "$tableau2"]}
```


Agrégation Avancée

Group avec Accumulateurs

```

1 db.collection.aggregate([
2   {
3     $group: {
4       _id: "$ville",
5       // Compter
6       nombre: {$sum: 1},
7
8       // Somme
9       totalAge: {$sum: "$age"},
10
11      // Moyenne
12      ageMoyen: {$avg: "$age"},
13
14      // Minimum
15      ageMin: {$min: "$age"},
16
17      // Maximum
18      ageMax: {$max: "$age"},
19
20      // Premier lment
21      premier: {$first: "$nom"},
22
23      // Dernier lment
24      dernier: {$last: "$nom"},
25
26      // Ajouter un tableau
27      tousLesNoms: {$push: "$nom"},
28
29      // Ajouter un ensemble
30      nomsUniques: {$addToSet: "$nom"},
31
32      // cart -type
33      ecartType: {$stdDevPop: "$age"},
34
35      // cart -type chantillon
36      ecartTypeEchantillon: {$stdDevSamp: "$age"}
37    }
38  }
39 ])
```

Group Multi-Niveaux

```

1 db.collection.aggregate([
2   {
3     $group: {
4       _id: {
5         ville: "$ville",
6         age: "$age"
7       },
8       nombre: {$sum: 1}
9     }
10  },
11  {
```

```

12     $group: {
13         _id: "$_id.ville",
14         groupes: {$push: {
15             age: "$_id.age",
16             nombre: "$nombre"
17         }}
18     }
19 }
20 ])
```

Pipeline Complexe - Exemple Complet

```

1 db.ventes.aggregate([
2     //  tape  1: Filtrer
3     {
4         $match: {
5             date: {$gte: ISODate("2024-01-01")},
6             montant: {$gt: 0}
7         }
8     },
9
10    //  tape  2: D plier les produits
11    {
12        $unwind: "$produits"
13    },
14
15    //  tape  3: Calculer le total par produit
16    {
17        $project: {
18            produitId: "$produits.id",
19            quantite: "$produits.quantite",
20            prixUnitaire: "$produits.prix",
21            total: {
22                $multiply: [
23                    "$produits.quantite",
24                    "$produits.prix"
25                ]
26            },
27            date: 1
28        }
29    },
30
31    //  tape  4: Regrouper par produit
32    {
33        $group: {
34            _id: "$produitId",
35            quantiteTotale: {$sum: "$quantite"},
36            revenuTotal: {$sum: "$total"},
37            nombreVentes: {$sum: 1},
38            prixMoyen: {$avg: "$prixUnitaire"}
39        }
40    },
41
42    //  tape  5: Trier par revenu
43    {
44        $sort: {revenuTotal: -1}
45    },
46 ])
```

```
46
47 //   tape 6: Limiter aux 10 premiers
48 {
49     $limit: 10
50 },
51
52 //   tape 7: Formater la sortie
53 {
54     $project: {
55         _id: 0,
56         produitId: "$_id",
57         quantiteTotale: 1,
58         revenuTotal: {
59             $round: ["$revenuTotal", 2]
60         },
61         nombreVentes: 1,
62         prixMoyen: {
63             $round: ["$prixMoyen", 2]
64         }
65     }
66 }
67 ])
```

Index et Performance

Création d'Index

```
1 // Index simple
2 db.collection.createIndex({nom: 1})
3
4 // Index compos
5 db.collection.createIndex({ville: 1, age: -1})
6
7 // Index unique
8 db.collection.createIndex({email: 1}, {unique: true})
9
10 // Index sur champ imbriqu
11 db.collection.createIndex({"adresse.ville": 1})
12
13 // Index sur tableau
14 db.collection.createIndex({tags: 1})
```

Analyse de Performance

```
1 // Expliquer une requête
2 db.collection.find({age: 30}).explain("executionStats")
3
4 // Analyser le plan d'exécution
5 db.collection.find({age: 30}).explain("queryPlanner")
```

Exemples Pratiques

Exemple 1 : Statistiques par Ville

```
1 db.personnes.aggregate([
2   {
3     $group: {
4       _id: "$ville",
5       nombre: {$sum: 1},
6       ageMoyen: {$avg: "$age"},
7       ageMin: {$min: "$age"},
8       ageMax: {$max: "$age"}
9     }
10  },
11  {
12    $sort: {nombre: -1}
13  }
14 ])
```

Exemple 2 : Top 5 des Produits les Plus Vendus

```
1 db.ventes.aggregate([
2   {
3     $unwind: "$produits"
4   },
5   {
6     $group: {
7       _id: "$produits.nom",
8       quantiteTotale: {$sum: "$produits.quantite"},
9       revenuTotal: {
10        $sum: {
11          $multiply: [
12            "$produits.quantite",
13            "$produits.prix"
14          ]
15        }
16      }
17    }
18  },
19  {
20    $sort: {quantiteTotale: -1}
21  },
22  {
23    $limit: 5
24  }
25 ])
```

Exemple 3 : Requête avec Jointure

```
1 db.commandes.aggregate([
2   {
3     $lookup: {
4       from: "clients",
5       localField: "clientId",
6       foreignField: "_id",
```

```
7         as: "client"
8     }
9 },
10 {
11     $unwind: "$client"
12 },
13 {
14     $project: {
15         numeroCommande: 1,
16         date: 1,
17         montant: 1,
18         clientNom: "$client.nom",
19         clientVille: "$client.ville"
20     }
21 }
22 ])
```

Exemple 4 : Calcul de Statistiques Mensuelles

```
1 db.ventes.aggregate([
2     {
3         $project: {
4             annee: {$year: "$date"},
5             mois: {$month: "$date"},
6             montant: 1
7         }
8     },
9     {
10        $group: {
11            _id: {
12                annee: "$annee",
13                mois: "$mois"
14            },
15            nombreVentes: {$sum: 1},
16            montantTotal: {$sum: "$montant"},
17            montantMoyen: {$avg: "$montant"}
18        }
19    },
20    {
21        $sort: {
22            "_id.annee": 1,
23            "_id.mois": 1
24        }
25    }
26 ])
```

Résumé des Opérations Essentielles

CRUD

Opération	Commande
Insert	<code>insertOne()</code> , <code>insertMany()</code>
Read	<code>find()</code> , <code>findOne()</code>
Update	<code>updateOne()</code> , <code>updateMany()</code>
Delete	<code>deleteOne()</code> , <code>deleteMany()</code>

Opérateurs de Filtrage

Opérateur	Description
<code>\$gt</code>	Plus grand que
<code>\$gte</code>	Plus grand ou égal
<code>\$lt</code>	Plus petit que
<code>\$lte</code>	Plus petit ou égal
<code>\$in</code>	Dans une liste
<code>\$nin</code>	Pas dans une liste
<code>\$and</code>	ET logique
<code>\$or</code>	OU logique
<code>\$regex</code>	Expression régulière

Stages d'Agrégation

Stage	Description
<code>\$match</code>	Filtrer les documents
<code>\$project</code>	Sélectionner/transformer les champs
<code>\$group</code>	Regrouper et agréger
<code>\$sort</code>	Trier les résultats
<code>\$limit</code>	Limiter le nombre de résultats
<code>\$skip</code>	Ignorer des résultats
<code>\$unwind</code>	Déplier un tableau
<code>\$lookup</code>	Effectuer une jointure
<code>\$addFields</code>	Ajouter des champs calculés
<code>\$count</code>	Compter les documents

Accumulateurs de Group

Accumulateur	Description
\$sum	Somme
\$avg	Moyenne
\$min	Minimum
\$max	Maximum
\$first	Premier élément
\$last	Dernier élément
\$push	Ajouter à un tableau
\$addToSet	Ajouter à un ensemble
\$stdDevPop	Écart-type population
\$stdDevSamp	Écart-type échantillon